

BDS701: PP IA 1

QUESTION 1

Explain SIMD and MIMD architectures and compare them.

ANSWER

1. Definition / Introduction

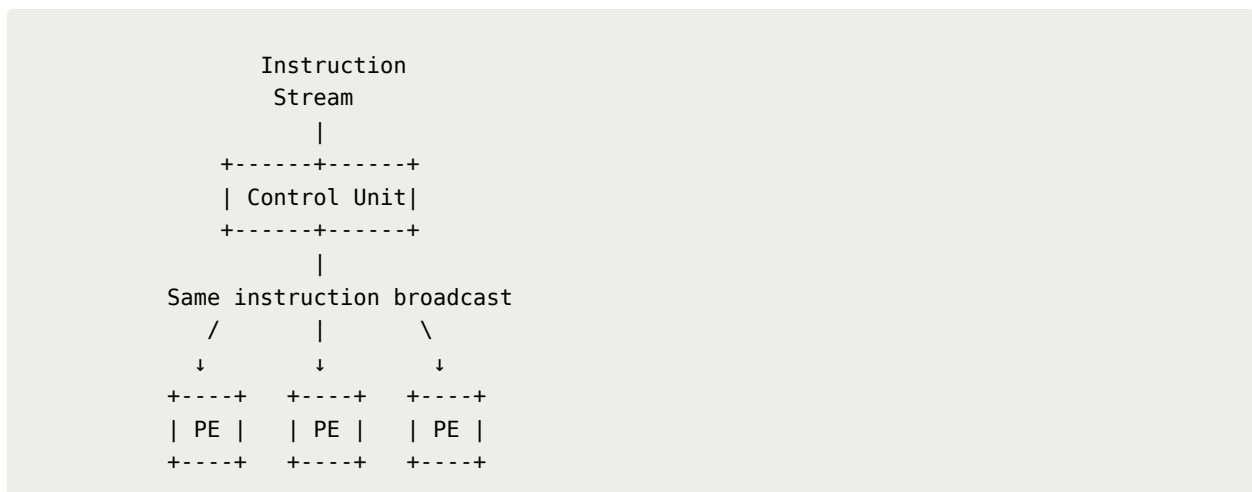
SIMD stands for **Single Instruction, Multiple Data**, while **MIMD** stands for **Multiple Instruction, Multiple Data**. They are parallel computer architectures classified according to the number of **instruction streams** and **data streams** they can handle simultaneously.

- **SIMD:** One instruction operates on multiple data items simultaneously.
- **MIMD:** Multiple instruction streams operate on multiple data streams concurrently.

2. SIMD Architecture

In a SIMD architecture, a **single control unit** broadcasts the same instruction to several **datapaths or processing elements**. Each datapath operates on a different portion of the data.

Diagram



| | |
Data 1 Data 2 Data 3

Diagram explanation:

- A single control unit generates the instruction.
- The same instruction is broadcast to multiple processing elements.
- Each processing element operates on a different data item.
- A datapath may remain idle if there is no matching data.

Example: Vector Addition

For two arrays **x** and **y**:

```
for (i = 0; i < n; i++)  
    x[i] += y[i];
```

Different datapaths can perform the same addition on different elements simultaneously.

If there are fewer datapaths than elements, the operation is performed in blocks. For example, with **4 datapaths and 15 elements**, the elements are processed as:

0-3 → 4-7 → 8-11 → 12-14

In the final group, one datapath remains idle.

Characteristics of SIMD

1. Single instruction stream.
2. Multiple data streams.
3. Single control unit with multiple datapaths.
4. Suitable for **data-parallel operations**.
5. Traditional SIMD systems operate synchronously.

6. Conditional branching can cause some datapaths to remain idle, reducing efficiency.

3. MIMD Architecture

In a MIMD architecture, multiple processors execute **different instructions on different data sets simultaneously**. Each processor is independent and has its own control unit and local datapath/memory.

Diagram

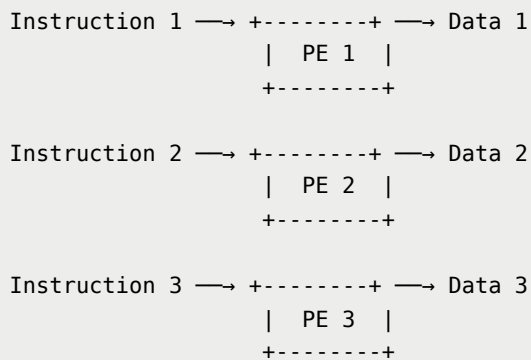


Diagram explanation:

- Each processing unit has its own instruction stream.
- Different processing units can execute different instructions.
- Each unit operates on its own data.
- The processors can operate independently and asynchronously.

Characteristics of MIMD

1. Multiple instruction streams.
2. Multiple data streams.
3. Independent control units.
4. Processors can execute at different rates.
5. No requirement for global clock or lockstep execution.

6. Supports both shared-memory and distributed-memory organizations.
7. Scales well with increasing processors.

Types of MIMD

1. Shared-Memory MIMD

- All processors share a common memory space.
- Processors communicate through shared memory.
- Synchronization mechanisms are required to prevent conflicts.
- Examples: multicore processors and symmetric multiprocessing systems.

2. Distributed-Memory MIMD

- Each processor has its own local memory.
- Processors communicate by passing messages through an interconnection network.
- Provides high scalability and is suitable for cluster and grid computing.

4. Comparison of SIMD and MIMD

Basis	SIMD	MIMD
Full form	Single Instruction, Multiple Data	Multiple Instruction, Multiple Data
Instruction streams	Single	Multiple
Data streams	Multiple	Multiple
Control unit	Single control unit	Independent control units
Execution	Same instruction on multiple data	Different instructions on different data
Synchronization	Required in traditional SIMD execution	Not required for processors to execute at the same pace
Parallelism	Mainly data parallelism	Supports different types of parallel work

Basis	SIMD	MIMD
Suitable for	Data-parallel applications	General-purpose and varied parallel applications
Example	GPUs, vector processors	Multicore processors, SMP systems, clusters
Scalability	—	Scales well with increasing processors

5. Limitations / Important Points

SIMD:

- Conditional branching may cause some processing elements to remain idle.
- Unequal division of data can leave datapaths underutilized.
- Traditional SIMD systems require synchronous operation.

MIMD:

- The architecture is more general and supports independent processing.
- MIMD systems can be organized as shared-memory or distributed-memory systems.

QUESTION 2

Explain different interconnection networks such as bus, ring, mesh, tree, and crossbar in brief.

ANSWER

1. Definition / Introduction

An **interconnection network** provides communication between processors, memory modules, and other components of a parallel computer. The interconnect has a decisive effect on system performance; even with high-performance processors and memory, a slow interconnect can seriously reduce the performance of a parallel program.

2. Bus Interconnection

A **bus** is a collection of parallel communication wires together with hardware that controls access to the bus. The communication wires are **shared by all devices** connected to the bus.

Diagram



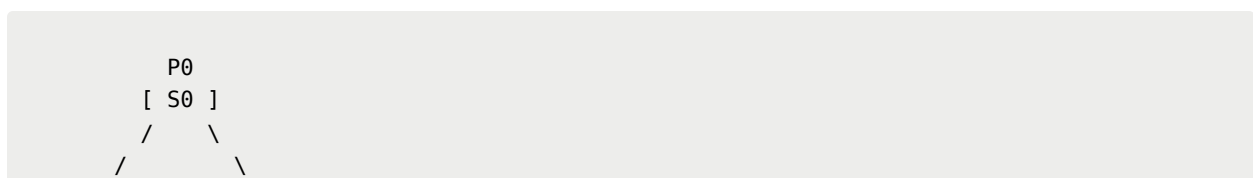
Important points:

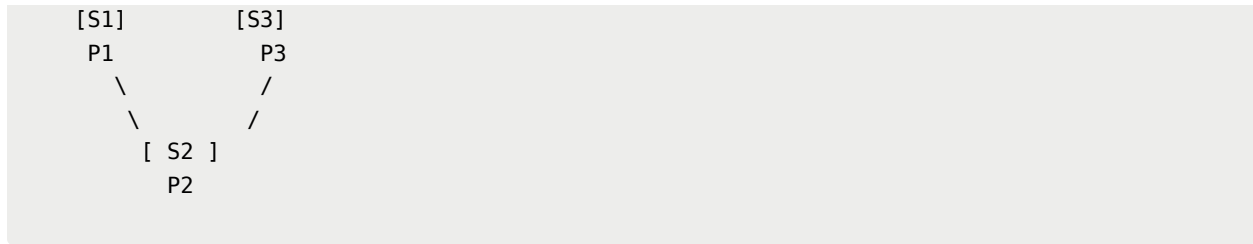
- Provides a simple connection between processors and memory.
- Has **low cost and flexibility**.
- Multiple devices can be connected with little additional cost.
- As the number of devices increases, **contention** for the shared bus increases.
- Consequently, expected performance decreases.
- Therefore, buses are suitable for systems with only a few processors.

3. Ring Interconnection

A **ring** is a direct interconnect in which the switches are connected in a circular arrangement. The links are bidirectional.

Diagram





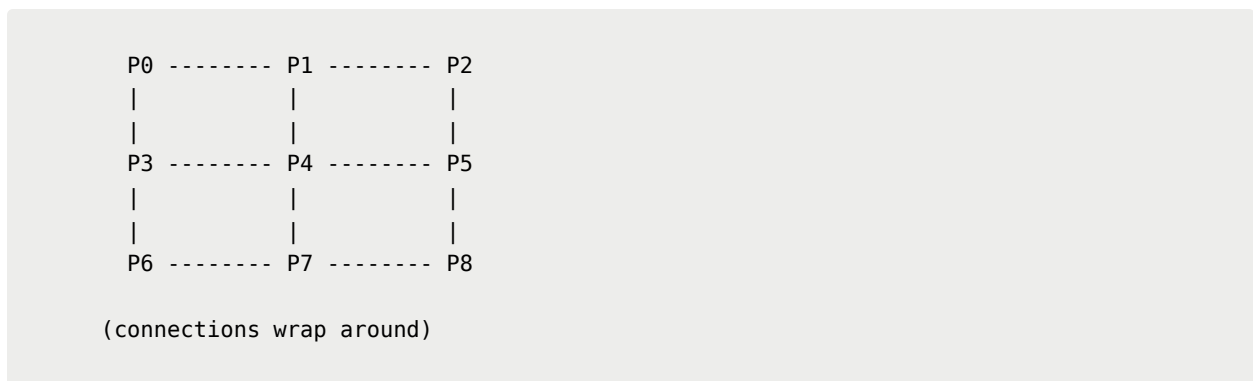
Important points:

- It is a **direct interconnect**.
- A ring is superior to a simple bus because it allows **multiple simultaneous communications**.
- However, certain communication patterns can require some processors to wait for other processors to complete their communications.

4. Mesh Interconnection

The **two-dimensional toroidal mesh** is a direct interconnect. In this structure, the switches must support multiple links, including wraparound connections.

Diagram



Important points:

- It is a **direct interconnect**.
- It provides more possible simultaneous communication patterns than a ring.
- It is more expensive than a ring because the switches are more complex.
- For a system with pp processors, the toroidal mesh has **$2p^2$ switch-to-switch links**, whereas a ring has pp links.

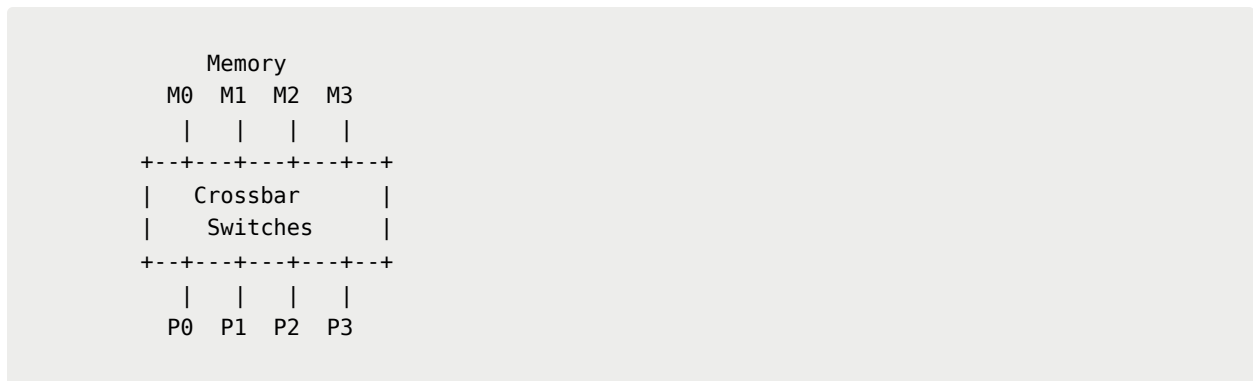
- For a square two-dimensional toroidal mesh, the bisection width is:

$$2 \sqrt{p}$$

5. Crossbar Interconnection

A **crossbar** is a relatively simple and powerful **switched interconnect**. Switches control the routing of data among connected devices.

Diagram



Important points:

- The links are bidirectional in the shared-memory crossbar described in the notes.
- A crossbar allows **simultaneous communication among different devices**.
- With at least as many memory modules as processors, a conflict occurs only when two cores simultaneously attempt to access the **same memory module**.
- Hence, crossbars are much faster than buses.
- Their major disadvantage is **high cost**, because the switches and links are relatively expensive.

6. Comparison

Network	Main characteristic	Advantage	Limitation
Bus	Shared communication wires	Low cost and flexible	Contention increases as devices increase

Network	Main characteristic	Advantage	Limitation
Ring	Processors/switches connected in a ring	Allows multiple simultaneous communications	Some communication patterns cause waiting
Mesh	Multiple links between neighboring switches	More simultaneous communication patterns than ring	More expensive than ring
Crossbar	Switched interconnect	Simultaneous communication; much faster than bus	High cost
Tree	—	—	Not sufficiently covered in the uploaded notes

7. Effect on Performance

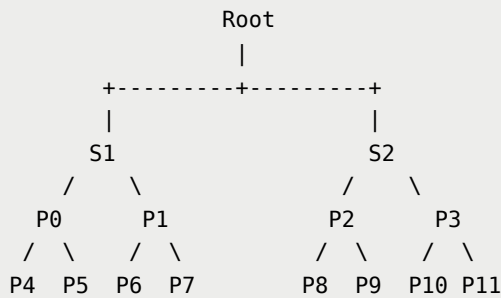
The choice of interconnection network directly affects parallel-system performance.

- A **bus** suffers from increasing contention as processors are added.
- A **ring** permits multiple simultaneous communications but can still create waiting for particular communication patterns.
- A **mesh** provides greater connectivity and more possible simultaneous communication patterns than a ring.
- A **crossbar** supports simultaneous communication among different devices and is therefore much faster than a bus, but at higher cost.

8. Tree Network

A **tree interconnection network** is a hierarchical network in which processing nodes are connected in the form of a **tree structure**. It consists of a root node, intermediate nodes, and leaf nodes. Communication takes place through the links connecting parent and child nodes.

Diagram



Working

- The **root** is at the top of the hierarchy.
- Intermediate switches/nodes connect the upper and lower levels.
- **Leaf nodes** generally represent processing elements or other end devices.
- Data travels through the hierarchical links between nodes.
- Communication between two leaf nodes may pass through their common parent and higher-level nodes.

Advantages

1. **Hierarchical organization** makes the network easy to structure.
2. It can support a relatively large number of processing nodes.
3. Nodes can be added by extending lower levels of the tree.
4. Communication paths are organized according to the hierarchy.

Limitations

1. Higher-level nodes and links can become **bottlenecks** because traffic from many lower-level nodes may pass through them.
2. Failure of an important upper-level node or link can affect communication between multiple lower-level nodes.
3. The performance can decrease when communication traffic becomes concentrated near the root.

QUESTION 3

Differentiate between shared-memory and distributed-memory architectures with suitable diagrams.

ANSWER

1. Definition / Introduction

There are two principal types of **MIMD systems**: **shared-memory systems** and **distributed-memory systems**.

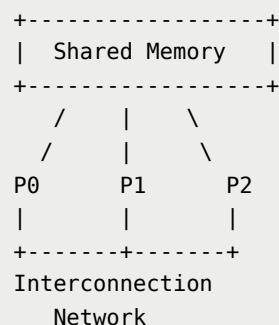
- In a **shared-memory system**, a collection of autonomous processors is connected to a memory system through an interconnection network, and **each processor can access each memory location**.
- In a **distributed-memory system**, each processor is paired with its own **private memory**, and the processor-memory pairs communicate through an interconnection network.

2. Shared-Memory Architecture

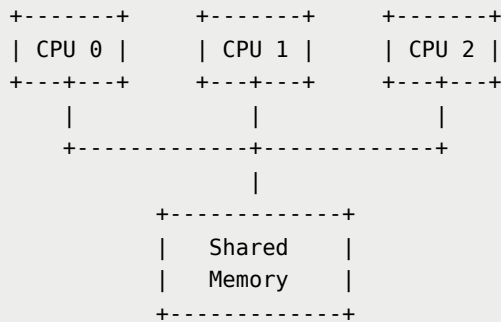
In a shared-memory system, all processors have access to a **common memory system**.

The processors usually communicate **implicitly** by accessing shared data structures.

Diagram



A more exam-friendly representation is:



Characteristics

1. Multiple processors/cores are connected to a common memory system.
2. Every processor can access every memory location.
3. Communication is generally through **shared data structures**.
4. Shared-memory systems commonly use one or more **multicore processors**.
5. Cores may have private level-1 caches, while other cache levels may or may not be shared.

Types of Shared Memory

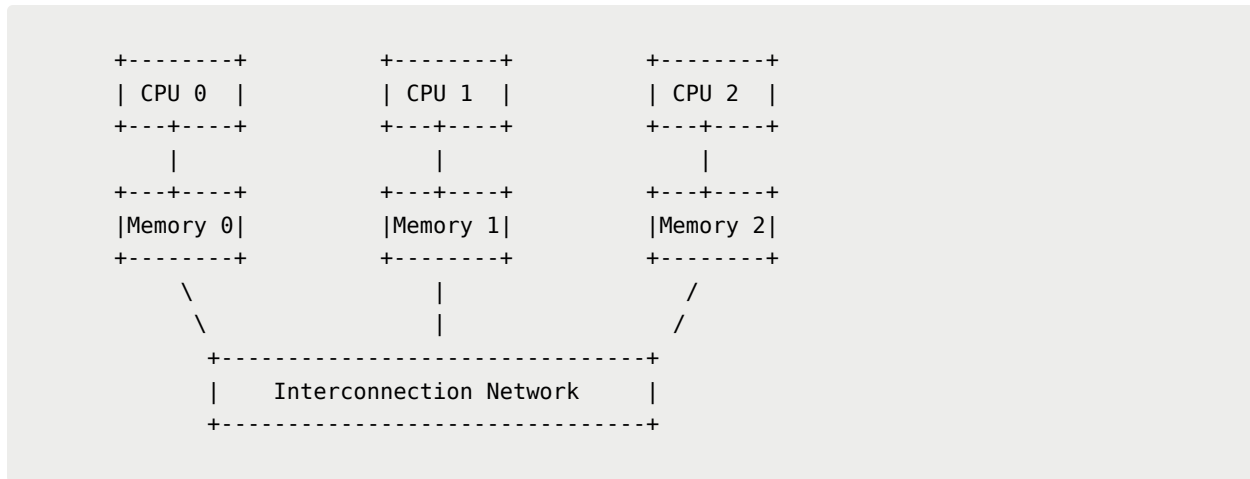
There are two types:

- **UMA — Uniform Memory Access:** all memory locations take the same amount of time to access.
- **NUMA — Nonuniform Memory Access:** a memory location directly connected to a processor can be accessed more quickly than a location that must be accessed through another chip.

3. Distributed-Memory Architecture

In a distributed-memory system, each processor is associated with its own **private memory**. The processor-memory pairs communicate over an **interconnection network**.

Diagram



Characteristics

1. Each processor has its own **private memory**.
2. A processor can directly access only its own memory.
3. Communication between processor-memory pairs takes place through an interconnection network.
4. Processors usually communicate **explicitly** by sending messages or using special functions that provide access to another processor's memory.
5. Distributed-memory programs are usually executed by starting **multiple processes** rather than multiple threads.
6. The most widely available distributed-memory systems are **clusters**, consisting of commodity systems connected by a commodity interconnection network.

4. Comparison of Shared-Memory and Distributed-Memory

Basis	Shared-Memory	Distributed-Memory
Memory organization	Common/shared memory	Private memory for each processor
Memory access	Each processor can access each memory location	Processor directly accesses its own private memory

Basis	Shared-Memory	Distributed-Memory
Communication	Usually implicit through shared data structures	Usually explicit through message passing/special functions
Processors	Autonomous processors connected to memory system	Processor-memory pairs connected by network
Programming unit	Usually multiple threads	Usually multiple processes
Memory sharing	Shared among processors	Distributed among processors
Scalability	Interconnect scaling is a major hardware issue	More suitable for large systems
Typical systems	Multicore/shared-memory systems	Clusters and other distributed-memory systems
Communication mechanism	Shared data structures	Message passing or remote-memory functions
Large-scale use	Limited by cost/scaling of shared interconnects	Can support systems with thousands of processors

5. Advantages and Limitations

Shared-Memory

Advantages:

1. Processors can access common data structures directly.
2. Communication is generally implicit through shared memory.
3. Shared-memory programming is often more appealing to programmers than explicitly sending messages.

Limitations:

1. Scaling the interconnect becomes difficult.
2. A bus is suitable only for systems with a relatively small number of processors because contention increases as processors are added.
3. Large crossbar interconnects are expensive.

Distributed-Memory

Advantages:

1. Distributed-memory interconnects such as the **hypercube and toroidal mesh** are relatively inexpensive.
2. Distributed-memory systems with **thousands of processors** have been built.
3. They are often better suited for problems requiring **vast amounts of data or computation**.

Limitation:

- Communication is explicit, generally requiring message passing or special communication functions.

6. Scalability

The principal hardware issue in comparing shared-memory and distributed-memory systems is the **cost of scaling the interconnect**.

In shared-memory systems:

```
More processors
  ↓
More contention for shared interconnect
  ↓
Difficult/expensive scaling
```

In distributed-memory systems:

```
More processor-memory pairs
  ↓
Distributed interconnection
  ↓
Better suitability for large systems
```

QUESTION 4

Explain the mechanisms used for coordinating processes and threads in parallel programs.

ANSWER

1. Definition / Introduction

In parallel programs, multiple **processes/threads** execute concurrently. For problems that cannot be solved simply by dividing the work independently, their activities must be **coordinated**.

The three important requirements:

1. **Coordinate the work** of processes/threads.
2. **Synchronize** the processes/threads.
3. Provide **communication** among processes/threads.

These requirements are often interrelated. In distributed-memory programs, communication can implicitly synchronize processes, while in shared-memory programs, synchronization is often used for communication among threads.

2. Coordination of Threads and Processes

A. Thread Coordination in Shared-Memory Programs

In shared-memory programs:

- Variables can be **shared or private**.
- Shared variables can be read or written by any thread.
- Private variables are ordinarily accessible only by one thread.
- Communication among threads is usually performed through **shared variables**, making communication implicit.

Threads can be organized using two approaches.

1. Dynamic Threads

- A **master thread** waits for work requests.

- When a request arrives, it **forks a worker thread**.
- The worker performs the requested work.
- After completing the work, the worker terminates and **joins the master thread**.
- This makes efficient use of system resources because thread resources are used while the thread is running.

2. Static Threads

- All threads are forked after the required setup by the master thread.
- The threads continue until all work is completed.
- The threads then join the master thread.
- The master can perform cleanup and terminate.
- It may use resources less efficiently when a thread is idle, but repeated forking and joining can itself be time-consuming.

3. Synchronization

Synchronization is required to control the execution of processes/threads when their operations depend on one another.

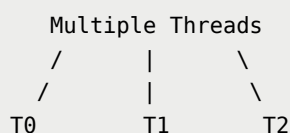
A. Critical Section and Mutual Exclusion

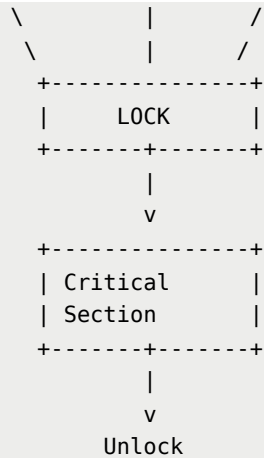
When two threads simultaneously access a shared resource and their accesses can produce an error, a **race condition** can occur.

A **critical section** is a block of code that can be executed by **only one thread at a time**.

Therefore, mutually exclusive access to a critical section must be ensured.

Diagram





4. Mutex Lock

The most commonly used mechanism for ensuring **mutual exclusion** is a **mutual exclusion lock (mutex)** or simply a lock.

Working of Mutex

1. A critical section is protected by a mutex.
2. Before entering the critical section, a thread must **obtain the mutex** using a mutex/lock function.
3. Only the thread that owns the mutex can execute the critical section.
4. Other threads attempting to enter the critical section wait in the lock function.
5. After completing the critical section, the thread **relinquishes the mutex** using an unlock function.

Example from the notes:

```
my_val = Compute_val(my_rank);

Lock(&add_my_val_lock);

x += my_val;

Unlock(&add_my_val_lock);
```

This ensures that only one thread at a time executes:

```
x += my_val
```

Important Point

A mutex **serializes the critical section** because only one thread can execute it at a time. Therefore, critical sections should be kept **as few and as short as possible**.

5. Busy-Waiting

An alternative to a mutex is **busy-waiting**.

In busy-waiting:

- A thread enters a loop whose purpose is to repeatedly test a condition.
- The thread remains in the loop until the required condition becomes true.
- Once the condition becomes true, the thread proceeds to the critical section.

Example:

```
if (my_rank == 1)
    while (!ok_for_1)
        ;                /* Busy-wait */

x += my_val;            /* Critical section */

if (my_rank == 0)
    ok_for_1 = true;
```

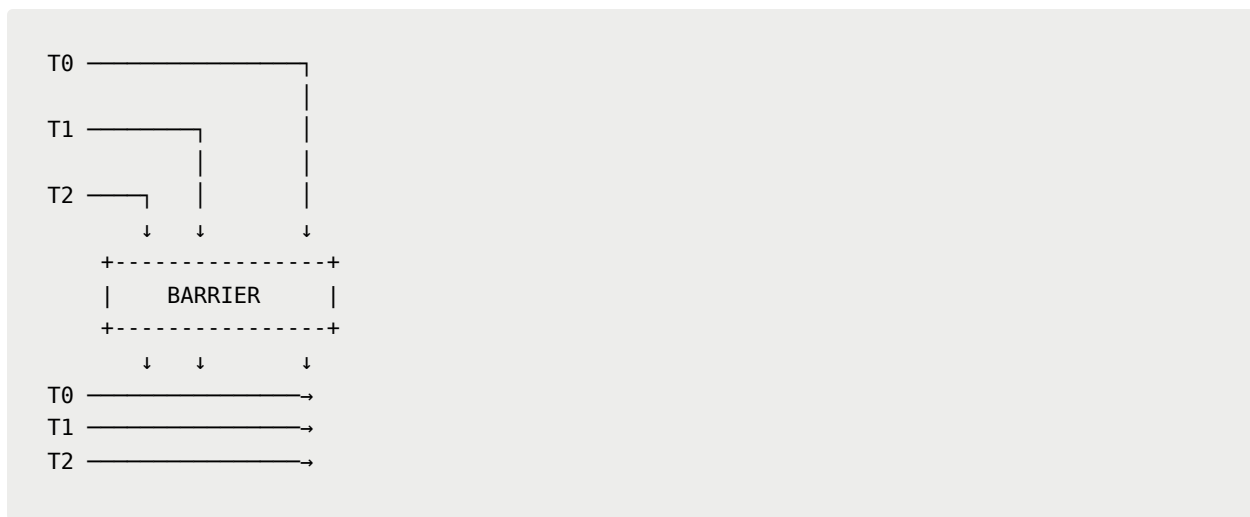
Here, thread 1 waits until thread 0 sets `ok_for_1` to true.

Limitation

Busy-waiting can be **wasteful of system resources**, because the core repeatedly checks the condition even though the thread is not performing useful work.

6. Barrier Synchronization

A **barrier** is used to synchronize processes/threads at a particular point in the program.



A process/thread calling the barrier waits until the required processes/threads have reached the barrier. The notes use `Barrier()` to approximately synchronize all processes/threads before continuing execution.

7. Communication Between Processes

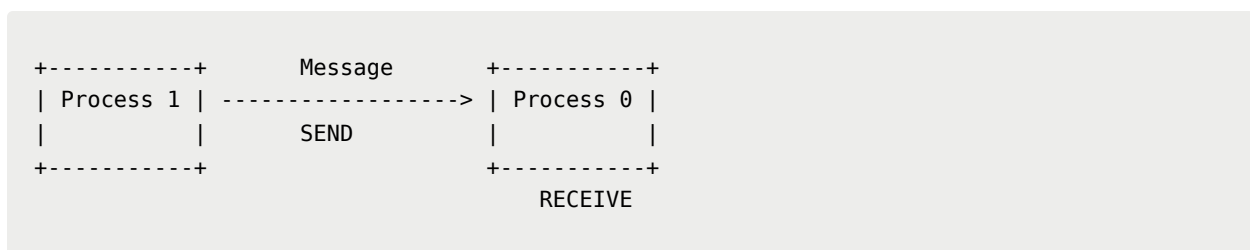
In **distributed-memory programs**, cores can directly access only their own private memories. Therefore, communication is normally performed using **message passing**.

Message Passing

A message-passing API provides, at minimum:

- **Send**
- **Receive**

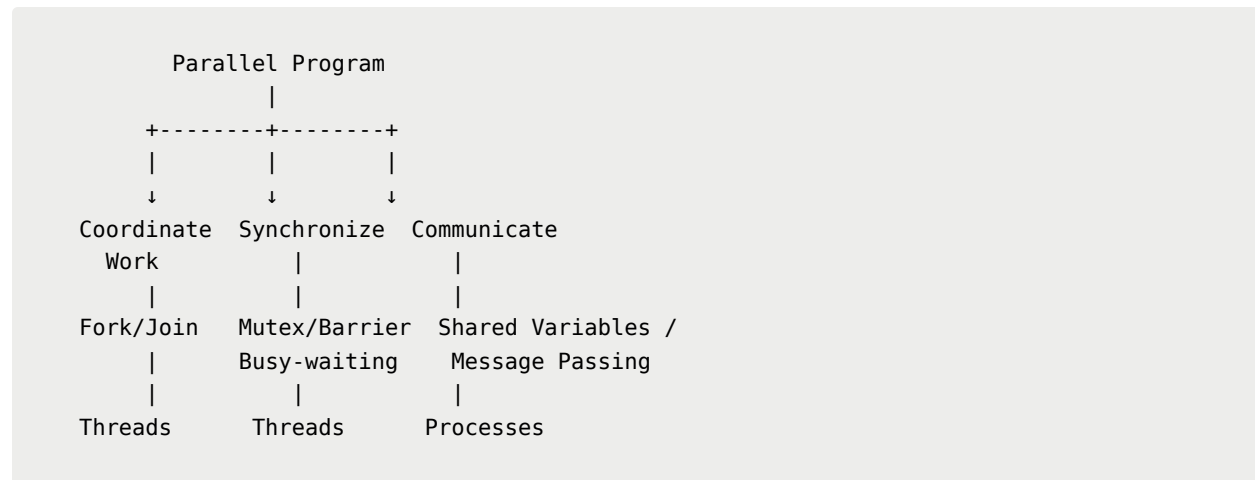
Processes typically identify one another using **ranks**.



Thus, processes communicate explicitly by sending and receiving messages.

8. Relationship Between the Mechanisms

The coordination mechanisms can be summarized as:



- **Fork/join** helps organize the execution of threads.
- **Mutex** provides mutual exclusion for critical sections.
- **Busy-waiting** allows a thread to wait for a condition.
- **Barrier** synchronizes processes/threads at a common point.
- **Shared variables** provide implicit communication in shared-memory programs.
- **Message passing** provides explicit communication in distributed-memory programs.

9. Important Consideration: Parallel Overhead

Coordination and synchronization introduce **parallel overhead**.

For example:

- Shared-memory programs may require critical sections and mutex operations.
- The mutex forces execution of the critical section to be serialized.

- Distributed-memory programs generally require transmission of data across the network, which is slower than local memory access.
- Such overheads generally increase as the number of processes/threads increases.

Therefore, coordination is necessary for correctness, but excessive synchronization can reduce parallel performance.

QUESTION 5

Discuss the relationship between speedup, efficiency, number of processors, and execution time in MIMD systems.

ANSWER

1. Definition / Introduction

The performance of an MIMD parallel program can be evaluated using **execution time, speedup, and parallel efficiency**. These quantities depend on the **problem size** and the number of processes/threads pp .

2. Execution Time

Let:

- T_{serial} = serial execution time
- $T_{parallel}$ = parallel execution time
- p = number of processes/threads

The ideal parallel execution time is:

$$T_{parallel} = \frac{T_{serial}}{p}$$

When this occurs, the program achieves **linear speedup**.

 BDS701-module-2-textbook(1)

In practice:

$$T_{parallel} = \frac{T_{serial}}{p} + T_{overhead}$$

where $T_{overhead}$ represents additional parallel overhead such as communication and mutual exclusion.

 BDS701-module-2-textbook(1)

3. Speedup

Speedup measures how much faster the parallel program runs compared with the serial program:

$$S = \frac{T_{serial}}{T_{parallel}}$$

For problem size n :

$$S(n, p) = \frac{T_{serial}(n)}{T_{parallel}(n, p)}$$

The ideal speedup is:

$$S = p$$

This is called **linear speedup**. In practice, perfect linear speedup is not usually achieved because of parallel overhead.  BDS701-module-3-textbook

4. Efficiency

Parallel efficiency is the **speedup per process**:

$$E = \frac{S}{p}$$

Therefore,

$$E = \frac{T_{serial}}{pT_{parallel}}$$

For linear speedup:

$$S = p$$

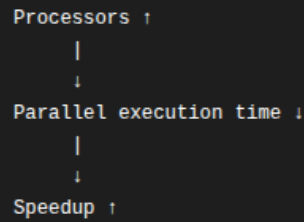
so:

$$E = \frac{p}{p} = 1$$

Thus, efficiency of **1.0** corresponds to ideal linear speedup.  BDS701-module-3-textbook

5. Relationship with Number of Processors

Ideally, increasing p should reduce execution time approximately by a factor of p :



However, in an actual MIMD program, increasing p also increases parallel overhead.

Examples:

- More threads may need to access a critical section.
- More processes may require more data to be transmitted across the network.
- Communication and synchronization introduce additional execution time.

BDS701-module-2-textbook(1)

Therefore, as p increases:

$$S < p$$

and generally:

$$E = \frac{S}{p} < 1$$

Efficiency tends to decrease as the number of processes increases when overhead grows.

BDS701-module-2-textbook(1)

6. Example

The notes give the following measured results:

p	Speedup S	Efficiency $E = S/p$
1	1.0	1.00
2	1.9	0.95
4	3.6	0.90
8	6.5	0.81
16	10.8	0.68

 BDS701-module-2-textbook(1)

For $p = 16$:

$$E = \frac{10.8}{16} = 0.675 \approx 0.68$$

Thus, although the number of processors increases from 8 to 16, the speedup does not double from 6.5 to 13. The reduction is due to parallel overhead.

7. Effect of Problem Size

Speedup and efficiency also depend on problem size.

The notes show that for the same program:

- Increasing problem size can increase speedup and efficiency.
- Decreasing problem size can decrease speedup and efficiency.
- For small p and large problem size, speedup can be close to linear.
- For large p and small problem size, speedup can  considerably less than p .

 BDS701-module-2-textbook(1)

QUESTION 6

Explain Amdahl's Law with suitable numerical examples. Analyze the effect of increasing the number of processors on parallel speedup.

ANSWER

1. Definition / Introduction

Amdahl's Law states that unless virtually all of a serial program is parallelized, the possible speedup is limited regardless of the number of cores available.

Let r be the fraction of the program that remains **unparallelized**, and $1-r$ the parallelizable fraction.

2. Speedup Equation

For p processors:

$$T_{parallel} = rT_{serial} + \frac{(1-r)T_{serial}}{p}$$

Therefore:

$$S = \frac{T_{serial}}{T_{parallel}} = \frac{1}{r + \frac{1-r}{p}}$$

As:

$$p \rightarrow \infty$$

the parallel portion approaches zero. Hence:

$$S_{max} = \frac{1}{r}$$

 BDS701-module-2-textbook

3. Numerical Example

Suppose **90%** of a program can be parallelized.

Therefore:

$$1 - r = 0.9$$

$$r = 0.1$$

Let:

$$T_{serial} = 20 \text{ seconds}$$

Then:

$$T_{parallel} = \frac{18}{p} + 2$$

and:

$$S = \frac{20}{18/p + 2}$$

BDS701-module-2-textbook

For $p = 10$:

$$S = \frac{20}{1.8 + 2} = \frac{20}{3.8} \approx 5.26$$

For $p = 100$:

$$S = \frac{20}{0.18 + 2} \approx 9.17$$

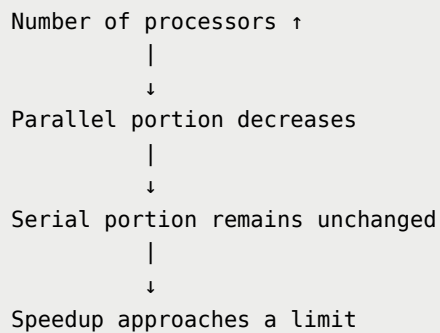
As p becomes very large:

$$S_{max} = \frac{1}{0.1} = 10$$

Thus speedup approaches **10**, but cannot exceed it.

BDS701-module-2-textbook

4. Effect of Increasing Processors



Therefore, increasing processors produces **diminishing returns** when a serial portion remains.

QUESTION 7

Explain scalability in MIMD systems. Discuss strong scaling and weak scaling with suitable examples and diagrams.

ANSWER

1. Definition

A program is approximately **scalable** if the problem size can be increased at an appropriate rate so that efficiency does not decrease as the number of processes increases.

There are two important cases:

- **Strong scalability**
- **Weak scalability**

2. Strong Scaling

A program is **strongly scalable** if the efficiency can be kept fixed when the number of processes/threads is increased **without increasing the problem size**.

Fixed Problem Size

```
|
+----- p = 2
|
+----- p = 4
|
+----- p = 8
|
+----- p = 16
```

Efficiency remains approximately constant

Thus, the same problem is solved faster as more processors are added.

3. Weak Scaling

A program is **weakly scalable** if efficiency can be kept fixed by increasing the problem size at the **same rate as the number of processes/threads**.

Processors	Problem Size
p	n
2	n
4	$2n$
8	$4n$
16	$8n$

Efficiency $\approx$ constant

4. Example

The notes provide an example where:

$$E = \frac{n}{625p}$$

for the specified range.

If:

$$n = 1000, \quad p = 2$$

then:

$$E = \frac{1000}{625(2)} = 0.8$$

If p is doubled to 4 while n remains 1000:

$$E = \frac{1000}{625(4)} = 0.4$$

But if both are doubled:

$$n = 2000, \quad p = 4$$

then:

$$E = \frac{2000}{625(4)} = 0.8$$

Thus increasing problem size together with processor count maintains efficiency in this example.

 BDS701-module-3-textbook

5. Strong vs Weak Scaling

Basis	Strong Scaling	Weak Scaling
Problem size	Fixed	Increased
Processor count	Increased	Increased
Efficiency	Remains fixed without increasing problem size	Remains fixed by increasing problem size at same rate
Objective	Solve same problem faster	Solve proportionally larger problem with similar efficiency

QUESTION 8

Explain how speedup and efficiency can be determined from measured execution times of an MIMD program. Discuss the factors that can cause variations in measured timings.

ANSWER

1. Speedup and Efficiency

Speedup is determined from the serial and parallel execution times:

$$S(n, p) = \frac{T_{\text{serial}}(n)}{T_{\text{parallel}}(n, p)}$$

Parallel efficiency is:

$$E(n, p) = \frac{S(n, p)}{p} = \frac{T_{\text{serial}}(n)}{pT_{\text{parallel}}(n, p)}$$

The ideal speedup is p , giving efficiency 1.0.  BDS701-module-3-textbook

2. Measuring Execution Time

For performance evaluation, the notes recommend measuring **elapsed or wall-clock time** for the part of the program of interest.

Conceptually:

```
start = current time

    Code to be measured

finish = current time

Elapsed time = finish - start
```

MPI provides:

MPI_Wtime

which returns wall-clock time.

3. Example

Suppose:

$$T_{serial} = 24 \text{ ms}$$

and:

$$T_{parallel} = 4 \text{ ms}, \quad p = 8$$

Then:

$$S = \frac{24}{4} = 6$$

and:

$$E = \frac{6}{8} = 0.75$$

Thus the speedup is **6** and the efficiency is **0.75**.  BDS701-module-2-textbook(1)

4. Factors Causing Variations in Measured Timings

1. Parallel overhead

Parallel programs introduce overhead such as:

- Mutual exclusion in shared-memory programs.

- Data transmission in distributed-memory programs.

Overhead generally increases as the number of processes/threads increases.

2. Communication and waiting

A process may have to wait for another process to execute a matching communication operation. This waiting contributes to actual elapsed time.

3. Timer resolution

The resolution of the timer determines the shortest event that can have a nonzero measured time. If the resolution is coarse, short events may not be measured accurately.

4. Problem size

Execution time, speedup and efficiency depend on problem size. Larger problems can produce different speedups and efficiencies than smaller problems.

5. Number of processes

Increasing the number of processes can increase communication and synchronization overhead, affecting measured execution time.

QUESTION 9

Explain Foster's methodology for designing parallel programmes with an example.

ANSWER

1. Definition / Introduction

Foster's methodology provides a systematic approach for designing parallel programs. The uploaded notes apply the methodology to the **parallel odd-even transposition sort**.

2. Main Stages

The methodology can be represented as:

```
Problem
↓
Partitioning
↓
Communication
↓
Agglomeration
↓
Mapping
↓
Parallel Program
```

3. Partitioning

The problem is divided into smaller **tasks** that can execute in parallel.

For odd-even transposition sort, the notes identify:

└ Determine the value of $a[i]$ at the end of phase j .

Thus, each task is associated with determining an element's value at a particular phase.

4. Communication

After identifying tasks, determine the communication required between them.

For odd-even transposition sort:

- The task determining $a[i]$ communicates with the task determining either $a[i - 1]$ or $a[i + 1]$.
- The value of $a[i]$ at the end of phase j must be available for determining its value at phase $j + 1$.

```
a[i-1] ↔ a[i] ↔ a[i+1]
      |
      ↓
    Next phase
```

5. Agglomeration

The identified tasks are grouped into larger units suitable for execution.

In the sorting example, each process is assigned:

$$\frac{n}{p}$$

keys at the beginning and end of execution. Therefore, aggregation and mapping are at least partially determined by the problem description.

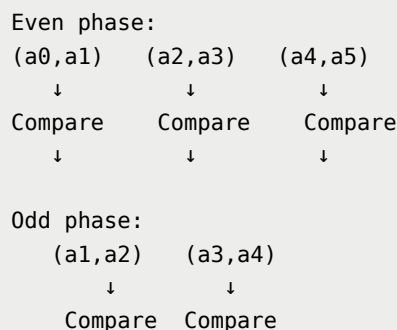
6. Mapping

The aggregated tasks are assigned to available processes.

For the parallel sorting example, the keys are distributed among the processes, with each process assigned n/p keys.

7. Example — Parallel Odd-Even Transposition Sort

Odd-even transposition sort has greater opportunities for parallelism than bubble sort because **all compare-swaps in a single phase can happen simultaneously**.



Thus, the methodology identifies independent compare-swap operations and the communications required between neighboring tasks.

QUESTION 10

Explain the role of `MPI_INIT`, `MPI_FINALISE`, `MPI_COMM_SIZE` and `MPI_COMM_RANK` using a program as an example. Define what is a communicator as well.

ANSWER

1. Communicator

In MPI, a **communicator** is a collection of processes that can send messages to each other.

After an MPI program is started, MPI creates a communicator containing all the processes started by the user. This communicator is called:

`MPI_COMM_WORLD`

2. `MPI_Init`

`MPI_Init` tells the MPI system to perform the necessary setup.

It may:

- Allocate storage for message buffers.
- Determine process ranks.

As a rule, no other MPI functions should be called before `MPI_Init`.

Syntax:

```
MPI_Init(&argc,&argv);
```

3. `MPI_Comm_size`

`MPI_Comm_size` returns the **number of processes in a communicator**.

```
MPI_Comm_size(MPI_COMM_WORLD,&comm_sz);
```

Here, `comm_sz` stores the number of processes in `MPI_COMM_WORLD`.

4. MPI_Comm_rank

`MPI_Comm_rank` returns the **rank of the calling process** in the communicator.

```
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
```

Process ranks normally range from:

0,1,2,...,p-1,2,\ldots,p-1

where pp is the number of processes.

5. MPI_Finalize

`MPI_Finalize` tells the MPI system that the program is finished using MPI and that MPI resources can be freed.

No MPI functions should generally be called after `MPI_Finalize`.

6. Example Program

```
#include <stdio.h>
#include <mpi.h>

int main(void) {
    int my_rank, comm_sz;

    MPI_Init(NULL, NULL);

    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    printf("Proc %d of %d\n", my_rank, comm_sz);

    MPI_Finalize();
}
```

```
    return 0;  
}
```

Working

```
MPI_Init  
  ↓  
MPI_COMM_WORLD created  
  ↓  
MPI_Comm_size → number of processes  
  ↓  
MPI_Comm_rank → rank of calling process  
  ↓  
Program execution  
  ↓  
MPI_Finalize
```

QUESTION 11

Explain message matching in MPI_Send and MPI_Recv. Make a note of the various parameters involved in this and what will happen when this is not done so.

ANSWER

1. Introduction

MPI uses **point-to-point communication** through `MPI_Send` and `MPI_Recv`.

A message sent by one process can be received by another only when the required **message-matching conditions** are satisfied.

2. Message Matching

Suppose process qq executes:

```
MPI_Send(send_buf_p, send_buf_sz, send_type, dest, send_tag, send_comm);
```

and process rr executes:

```
MPI_Recv(recv_buf_p, recv_buf_sz, recv_type, src, recv_tag, recv_comm, &status);
```

The message matches when:

1. `recv_comm = send_comm`
2. `recv_tag = send_tag`
3. `dest = r`
4. `src = q`

The send and receive buffers must also be compatible.

3. Buffer Compatibility

The first three pairs of arguments describe the data:

- `send_buf_p / recv_buf_p`
- `send_buf_sz / recv_buf_sz`
- `send_type / recv_type`

A common sufficient condition is:

$recv_type = send_type$

and:

$recv_buf_sz \geq send_buf_sz$

Then the message can be successfully received. [BDS701-module-3-textbook](#)

4. Message Matching Diagram

Process q		Process r
MPI_Send(...)	----->	MPI_Recv(...)

+-- destination = r	+-- source = q
+-- tag = X	+-- tag = X
+-- communicator = C	+-- communicator = C
+-- compatible buffer ----->	+-- compatible buffer

5. What Happens if Matching Is Incorrect?

`MPI_Recv` is **blocking**. If a process tries to receive a message and there is no matching send, it can **block forever**, causing the process to hang.

Possible errors include:

- Tags do not match.
- Source and destination ranks are incorrect.
- Communicators do not match.
- Buffer specifications are incompatible.

If a send blocks and there is no matching receive, the sending process can also hang.

If a send is buffered and there is no matching receive, the message can be lost.

6. Non-overtaking Rule

MPI requires messages to be **non-overtaking**.

If process qq sends two messages to process rr, the first message sent by qq must be available to rr before the second message.

However, messages sent from different processes do not have the same ordering requirement.

QUESTION 12

Explain the various parameters used in the functions `MPI_Send` and `MPI_Recv`.

ANSWER

1. MPI_Send

The syntax is:

```
int MPI_Send(void* msg_buf_p, int msg_size, MPI_Datatype msg_type, int dest, int tag, MPI_Comm communicator);
```

The arguments describe the **contents of the message and its destination**.

Parameters of MPI_Send

Parameter	Meaning
<code>msg_buf_p</code>	Points to the block of memory containing the message
<code>msg_size</code>	Number of objects in the message
<code>msg_type</code>	MPI datatype of the objects
<code>dest</code>	Rank of the destination process
<code>tag</code>	Identifies the message
<code>communicator</code>	Specifies the communication universe

The predefined datatypes is `MPI_INT`, `MPI_FLOAT`, `MPI_DOUBLE`, `MPI_CHAR`, etc.

2. MPI_Recv

The syntax is:

```
int MPI_Recv(void* msg_buf_p, int buf_size, MPI_Datatype buf_type, int source, int tag, MPI_Comm communicator, MPI_Status* status_p);
```

Parameters of MPI_Recv

Parameter	Meaning
<code>msg_buf_p</code>	Points to memory available for receiving the message
<code>buf_size</code>	Number of objects that can be stored

Parameter	Meaning
<code>buf_type</code>	Datatype of the receiving objects
<code>source</code>	Rank of the process from which the message is received
<code>tag</code>	Tag that should match the sending message
<code>communicator</code>	Communicator that must match the sender's communicator
<code>status_p</code>	Status information for the received message

3. Message Matching

The important matching relationships are:

MPI_Send		MPI_Recv
<code>dest</code>	----->	<code>source</code>
<code>send_tag</code>	----->	<code>tag</code>
<code>send_comm</code>	----->	<code>communicator</code>
<code>send buffer</code>	----->	<code>receive buffer</code>

For a successful match:

$dest = r, \quad source = q$

$send_tag = recv_tag$

$send_comm = recv_comm$

and the send and receive buffers must be compatible.

[BD5701-module-3-textbook](#)

4. Status Parameter

The `status_p` parameter can provide information about the received message.

After `MPI_Recv`, the sender and tag can be obtained using:

```
status.MPI_SOURCEstatus.MPI_TAG
```

The amount of received data can be obtained using `MPI_Get_count`.

If status information is not needed, we use:

```
MPI_STATUS_IGNORE
```

as the final argument.

5. Blocking Behavior

`MPI_Recv` is **blocking**: it does not return until the message has been received or an error has occurred.

`MPI_Send` can either block or buffer the message depending on the MPI implementation.

6. Example

```
MPI_Send(&local_int, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD); MPI_Recv  
(&local_int, 1, MPI_DOUBLE, source, 0, MPI_COMM_WORLD, MPI_STATUS_I  
GNORE);
```

Here:

- `&local_int` → message/receive buffer
 - `1` → number of objects
 - `MPI_DOUBLE` → datatype
 - `0 / source` → destination/source rank
 - `0` → tag
 - `MPI_COMM_WORLD` → communicator
 - `MPI_STATUS_IGNORE` → status information ignored
-